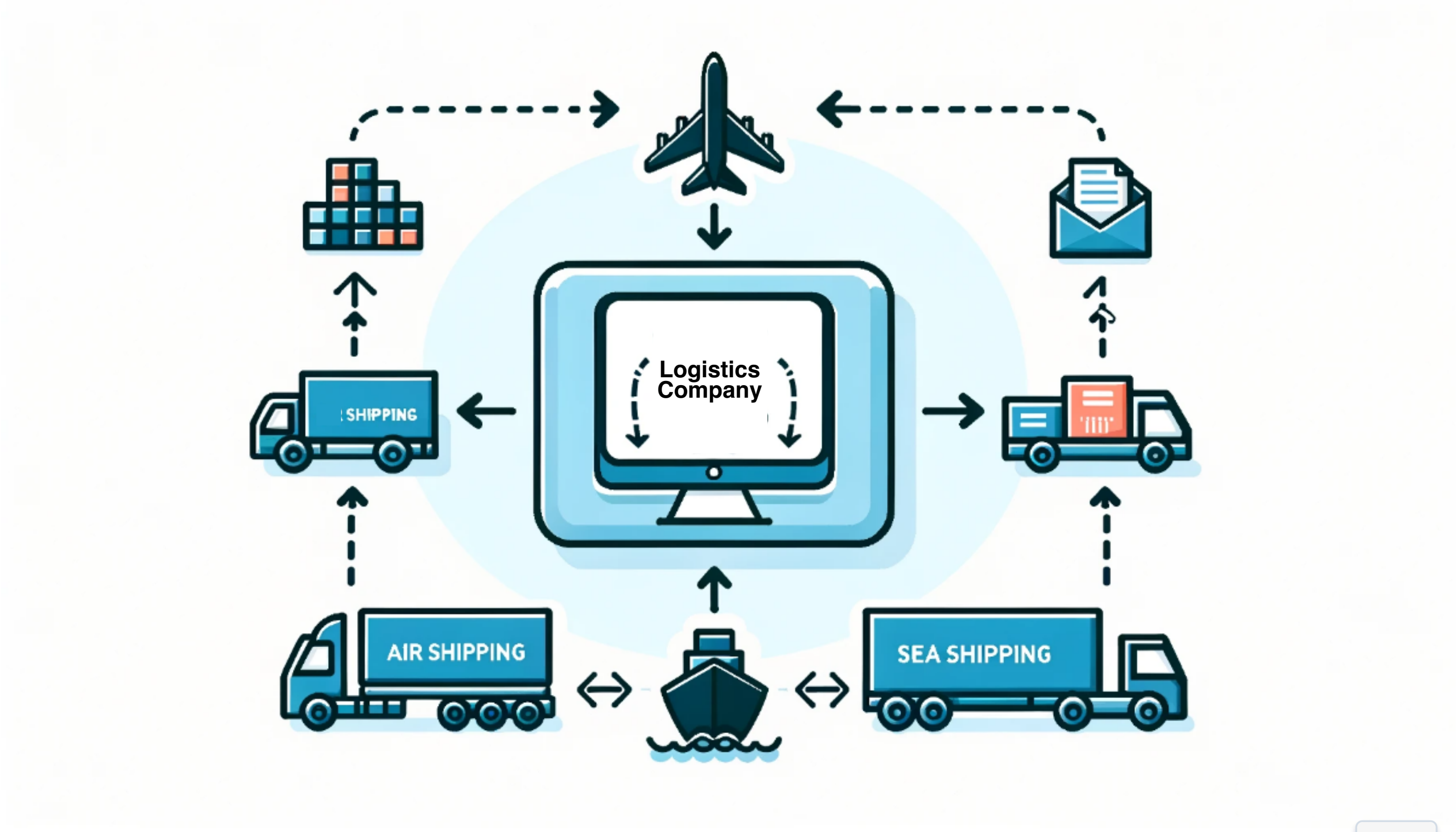
Strategy Pattern

The Strategy design pattern is a behavioral design pattern that allows you to change the behavior of an algorithm at runtime. Instead of explicitly implementing a single method, code receives run-time instructions describing which among a family of algorithms to use.

Consider a logistics firm that needs to determine the cost of shipping products. The shipping cost is determined by several elements, including the destination, weight, and delivery method (air, road, or sea). Putting all of these computations into one class would result in a complicated, difficult-to-maintain system, particularly if additional shipping options or price adjustments are made.



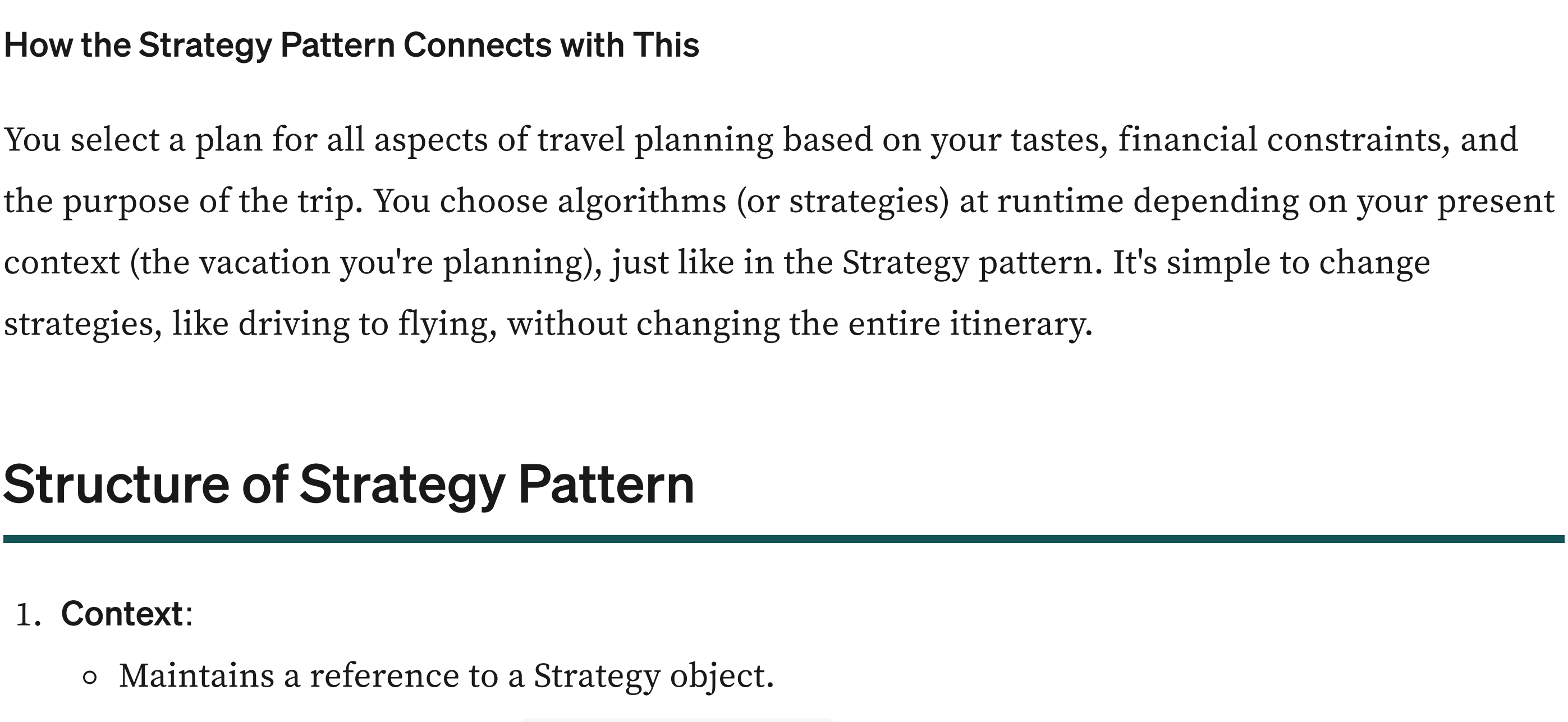
Strategy Pattern - Logistics Company Example

The shipping cost calculation strategies should be divided into distinct classes that each implement the same interface according to the strategy pattern. Next, without knowing which particular method is being used, the logistics system refers to this interface to determine the shipping cost. This method makes it simple to add new strategies without modifying the current code and enables dynamic changes to the shipping cost calculation strategy based on requirements.

Real-World Example

Consider organizing a trip. For every part of your trip, including accommodation, activities, and transportation, you have a number of options. Each choice of travel planning can be viewed as a strategy.

- Transportation Plan
  - Air Travel: Long-distance travel is quickest by air but is also more costly.
  - Train Travel: Beautiful routes, affordable, perfect for city-to-city travel.
  - Road travel is adaptable and perfect for quick trips or when you want to go at your own speed.
- Accommodations Strategy
  - Hotels: Cozy customer-focused hotels with a range of amenities.
  - Hostels: Social setting at an affordable price.
  - Vacation rentals: ideal for families or groups, they offer privacy and a homey feel.
- Activities
  - Adventure Sports: For those seeking extreme excitement, such as skydiving and scuba diving.
  - Cultural tours: seeing historical sites, museums, and local cultures.
  - Relaxation: Spas, beaches, and leisure pursuits are places to unwind.



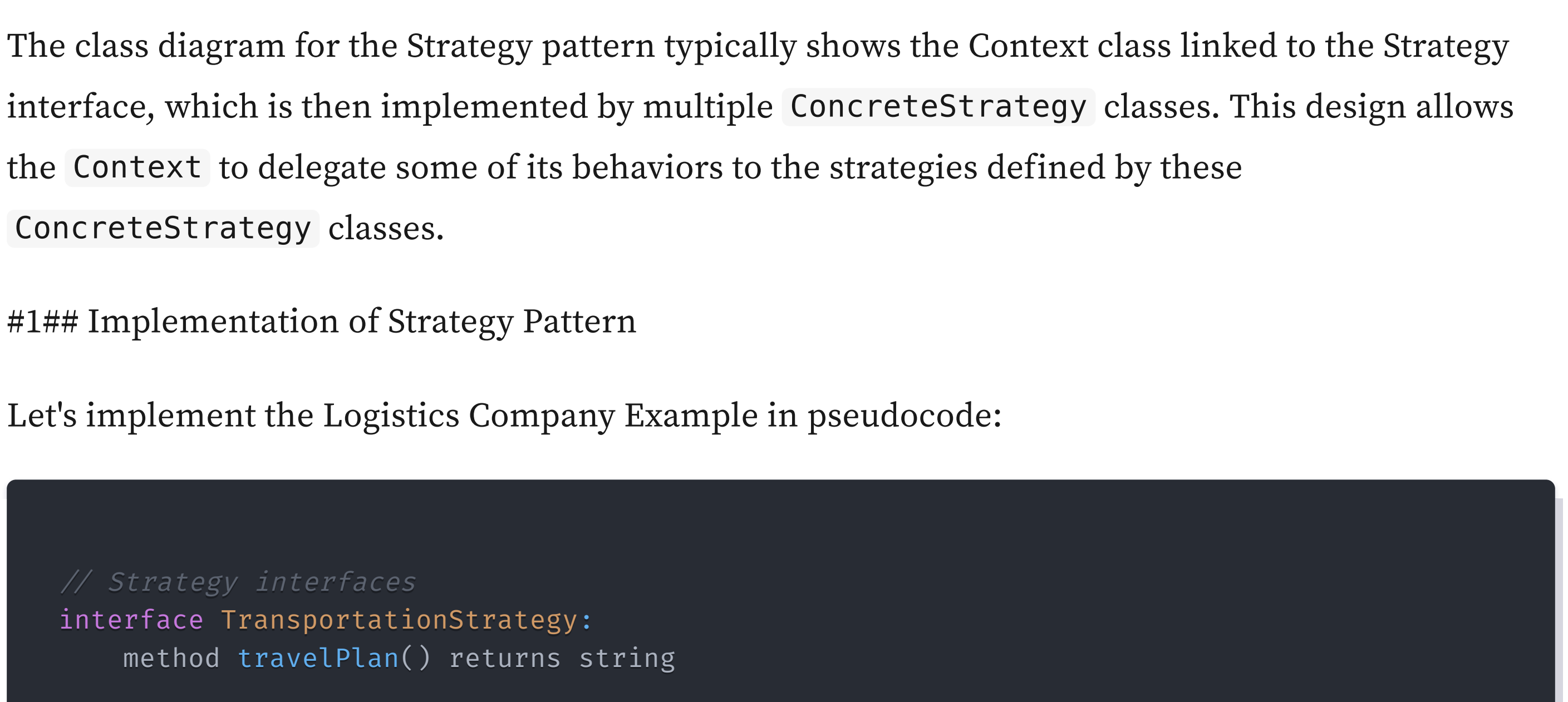
Strategy Pattern - Real-life Example

How the Strategy Pattern Connects with This

You select a plan for all aspects of travel planning based on your tastes, financial constraints, and the purpose of the trip. You choose algorithms (or strategies) at runtime depending on your present context (the vacation you're planning), just like in the Strategy pattern. It's simple to change strategies, like driving to flying, without changing the entire itinerary.

Structure of Strategy Pattern

1. **Context**:
   - Maintains a reference to a Strategy object.
   - Can be configured with a `ConcreteStrategy` object.
   - May define an interface that lets `Strategy` access its data.
2. **Strategy Interface**:
   - Declares an interface common to all supported algorithms. Context uses this interface to call the algorithm defined by a `ConcreteStrategy`.
3. **Concrete Strategies**:
   - Implements the algorithm using the `Strategy` interface.
   - Provides various alternatives for the behavior encapsulated in the `Context`.



Strategy Pattern - Class Diagram

The class diagram for the Strategy pattern typically shows the Context class linked to the Strategy interface, which is then implemented by multiple `ConcreteStrategy` classes. This design allows the `Context` to delegate some of its behaviors to the strategies defined by these `ConcreteStrategy` classes.

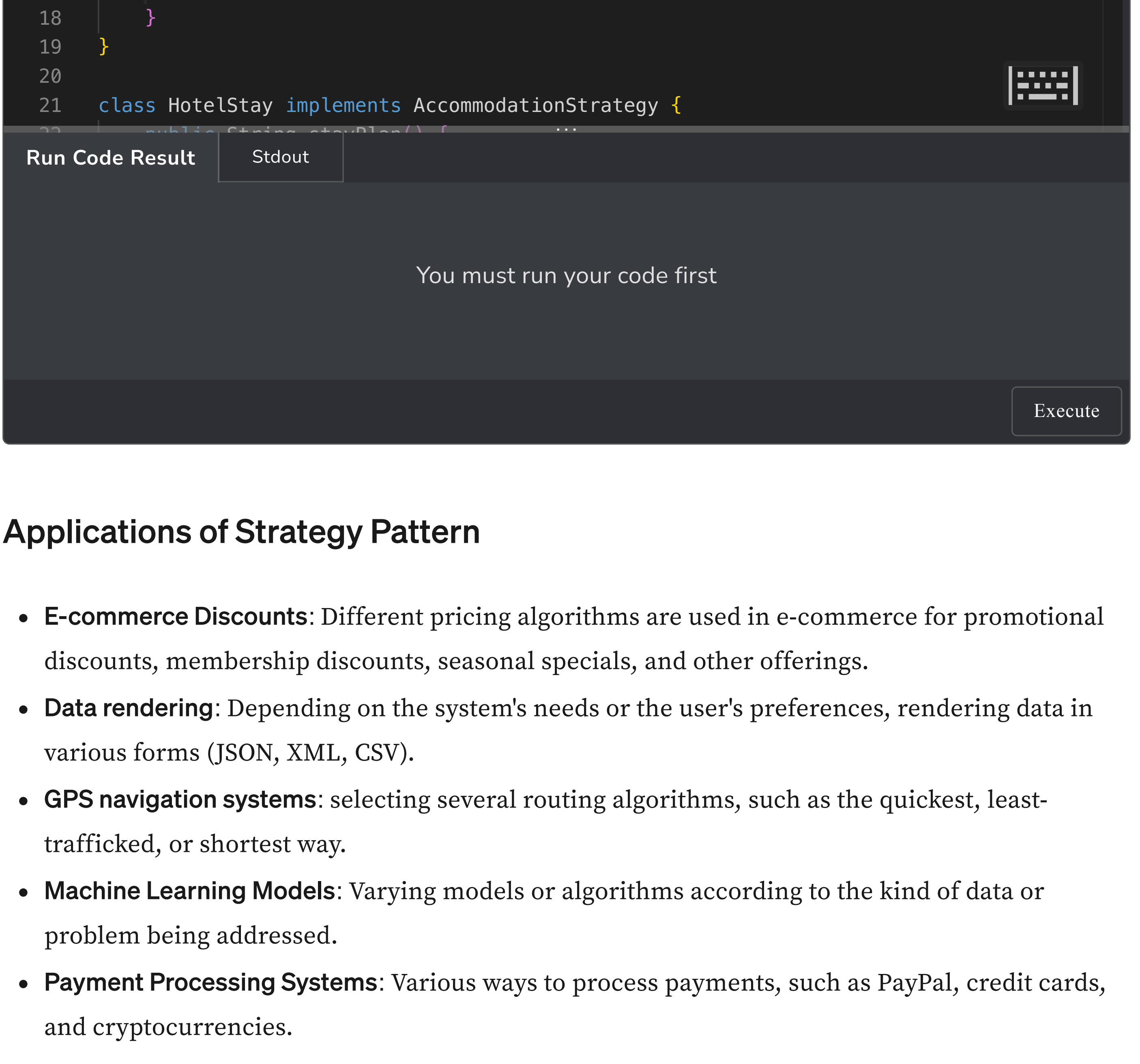
#1## Implementation of Strategy Pattern

Let's implement the Logistics Company Example in pseudocode:

```
// Strategy interfaces
interface TransportationStrategy:
    method travelPlan() returns string

interface AccommodationStrategy:
    method stayPlan() returns string

interface ActivityStrategy:
    method activityPlan() returns string

// Concrete Strategies
class AirTravel implements TransportationStrategy:
    method travelPlan() returns string:
        return "Travel by air: Fast and convenient for long distances."

class TrainTravel implements TransportationStrategy:
    method travelPlan() returns string:
        return "Travel by train: Enjoy scenic routes and comfortable travel."

class RoadTravel implements TransportationStrategy:
    method travelPlan() returns string:
        return "Travel by road: Flexible and ideal for exploration."

class HotelStay implements AccommodationStrategy:
    method stayPlan() returns string:
        return "Stay in a hotel: Enjoy comfort and luxury services."

class HostelStay implements AccommodationStrategy:
    method stayPlan() returns string:
        return "Stay in a hostel: Budget-friendly and social environment."

class VacationRentalStay implements AccommodationStrategy:
    method stayPlan() returns string:
        return "Stay in a vacation rental: Privacy and a homely feel."

class AdventureSports implements ActivityStrategy:
    method activityPlan() returns string:
        return "Engage in adventure sports: Exciting and thrilling experiences."

class CulturalTours implements ActivityStrategy:
    method activityPlan() returns string:
        return "Go on cultural tours: Explore local culture and historical sites."

class RelaxationActivities implements ActivityStrategy:
    method activityPlan() returns string:
        return "Relax at beaches or spas: Leisure and tranquility."

// Context Class: Travel Plan
class TravelPlan:
    transportationStrategy: TransportationStrategy
    accommodationStrategy: AccommodationStrategy
    activityStrategy: ActivityStrategy

    method setTransportationStrategy(TransportationStrategy strategy):
        this.transportationStrategy = strategy

    method setAccommodationStrategy(AccommodationStrategy strategy):
        this.accommodationStrategy = strategy

    method setActivityStrategy(ActivityStrategy strategy):
        this.activityStrategy = strategy

    method generatePlan() returns string:
        return transportationStrategy.travelPlan() + "\n" +
               accommodationStrategy.stayPlan() + "\n" +
               activityStrategy.activityPlan()

//Usage:
travelPlan = new TravelPlan()

// Setting different strategies
travelPlan.setTransportationStrategy(new AirTravel())
travelPlan.setAccommodationStrategy(new HotelStay())
travelPlan.setActivityStrategy(new CulturalTours())

// Generate and print the travel plan
print(travelPlan.generatePlan())
```

- **Strategy Interfaces** These interfaces specify a contract for three main categories of strategies: accommodation, activities, and transportation. Concrete strategy classes will carry out the methods associated with each strategy.
- **Concrete Strategy Classes** These classes implement the corresponding strategy interfaces. Each class implements the strategy differently. For example, `AirTravel` uses its own version of the `travelPlan()` method to implement the TransportationStrategy. Other strategies, such as `HotelStay`, `RoadTravel`, `TrainTravel`, and so on, are also implemented. Each strategy offers a different version of the method based on the strategy interface it implements.
- **Travel Plan** (**Context Class**) This class, TravelPlan, is the context in which the strategies will be used. It has references to all different kinds of techniques but does not really implement any of them. It contains methods for configuring the strategies (such as `setTransportationStrategy`) and a method for generating a travel plan based on the current set of strategies.
- **Usage** In order to use this pattern, you must first create an instance of the `TravelPlan` context, define the appropriate strategies (such as `HotelStay`, `AirTravel`, etc.), and then call the `generatePlan` method to obtain a compiled travel plan.

Implementations



```
// Strategy Interfaces
interface TransportationStrategy {
    String travelPlan();
}

interface AccommodationStrategy {
    String stayPlan();
}

interface ActivityStrategy {
    String activityPlan();
}

// Concrete Strategies
class AirTravel implements TransportationStrategy {
    public String travelPlan() {
        return "Travel by air: Fast and convenient for long distances.";
    }
}

class HotelStay implements AccommodationStrategy {
```

You must run your code first

Applications of Strategy Pattern

- **E-commerce Discounts**: Different pricing algorithms are used in e-commerce for promotional discounts, membership discounts, seasonal specials, and other offerings.
- **Data rendering**: Depending on the system's needs or the user's preferences, rendering data in various forms (JSON, XML, CSV).
- **GPS navigation systems**: selecting several routing algorithms, such as the quickest, least-trafficked, or shortest way.
- **Machine Learning Models**: Varying models or algorithms according to the kind of data or problem being addressed.
- **Payment Processing Systems**: Various ways to process payments, such as PayPal, credit cards, and cryptocurrencies.

Pros and Cons

Pros	Cons
Flexibility: Easily switch between different algorithms or strategies at runtime.	**Complexity:** Can increase the complexity of the code with multiple strategy classes.
Scalability: Simplifies adding new strategies without modifying the existing code.	**Overhead:** Additional overhead in terms of memory and runtime, especially with a large number of strategies.
Maintainability: Changes in algorithms or strategies don't affect the client code, improving maintainability.	**Understanding:** Requires a good understanding of each strategy to use it effectively.
Decoupling: Separates the concerns by decoupling algorithm implementation from the business logic.	**Initial Setup:** More upfront work is needed to define strategy interfaces and concrete classes.
Testability: Easier to unit test each strategy independently.	**Multiple Objects:** May lead to an increased number of objects in the system.

The Strategy pattern is a versatile tool in the toolkit of a developer, providing flexibility and maintainability in cases where numerous solutions for one specific task exist. It is particularly effective in systems where runtime behavior changes are required. By understanding and using this pattern, software design can be significantly enhanced, leading to easier extensions and modifications in the future.

← Previous
State Pattern
Next →
Template Method Pattern
☐ Mark as Completed